MATCOM, UH

Asignatura: Inteligencia Artificial

Jugador Virtual de Hex: Implementación de MCTS y MinMax con Heurísticas

Sebastian González Alfonso

sebagonz106@gmail.com

C312

21 de marzo de 2026

Resumen

El juego Hex es un juego de mesa estratégico combinatorio de dos jugadores que se disputa en un tablero con celdas hexagonales. A pesar de su aparente simplicidad en las reglas, el juego presenta un espacio de búsqueda de complejidad computacional significativa que lo convierte en un banco de pruebas interesante para algoritmos de búsqueda y toma de decisiones en inteligencia artificial. Este trabajo se enfoca en la implementación y análisis experimental de estrategias progresivamente mejoradas de Monte Carlo Tree Search, incorporando técnicas complementarias de evaluación rápida de acciones, reciclaje de árbol de búsqueda, paralelización y análisis de tableros reducidos. Se evalúa críticamente cada estrategia bajo restricción temporal estricta de 5 segundos por movimiento, resultando en la identificación de MCTS con evaluación rápida de acciones como el paradigma más efectivo bajo las restricciones del proyecto.

Palabras clave: *Hex, Monte Carlo Tree Search, juegos combinatorios, algoritmos de búsqueda, inteligencia artificial*

Abstract

Hex is a two-player strategic combinatorial board game played on a board with hexagonal cells. Despite its apparent simplicity in rules, the game presents a search space with significant computational complexity that makes it an interesting testbed for search algorithms and decision-making in artificial intelligence. This work focuses on the implementation and experimental analysis of progressively improved Monte Carlo Tree Search strategies, incorporating complementary techniques such as rapid action value estimation, search tree recycling, parallelization, and reduced board analysis. Each strategy is critically evaluated under strict time constraint of 5 seconds per move, resulting in identification of MCTS with rapid action value estimation as the most effective paradigm under the project constraints.

Keywords: *Hex, Monte Carlo Tree Search, combinatorial games, search algorithms, artificial intelligence*

Índice

1. Descripción del Juego Hex **4**
1.1. Fundamentales y Reglas del Juego 4
1.2. Propiedades Matemáticas y Complejidad Computacional 5
1.3. Geometría Hexagonal y Representación del Tablero 5

2. Algoritmos de Búsqueda **6**
2.1. Minimax 6
2.1.1. Desarrollo Histórico 6
2.1.2. Principios Fundamentales 6
2.1.3. Poda Alfa-Beta 6
2.2. Monte Carlo Tree Search 7
2.2.1. Contexto Histórico y Motivación 7
2.2.2. Principios Fundamentales 7
2.2.3. Fórmula UCT y Balance Exploración-Explotación 7

3. Implementación y Estrategias Experimentales **8**
3.1. BasicMCTSPlayer: MCTS Convencional 8
3.2. RecyclerMCTSPlayer: MCTS con Reciclaje de Árbol 9
3.3. RAVEMCTSPlayer: MCTS con Evaluación Rápida de Acciones 9
3.4. ProgressiveMCTSPlayer: MCTS con Estrategias Progresivas Adaptativas . . 10
3.5. ParallelizedMCTSPlayer: MCTS Paralelizado con Sincronización Ordenada . 10
3.6. ReducedBoardMCTSPlayer: MCTS con Tableros Reducidos mediante Fillin . 11
3.7. Noobex: Jugador Final 11

4. Análisis del Jugador Final Noobex **12**
4.1. Descripción de SmartPlayer 12
4.2. Ventajas 13
4.3. Limitaciones 13
4.4. Recomendación de Uso 13

5. Resultados Experimentales **13**

6. Investigación Futura **14**

7. Conclusiones **16**

Introducción

El problema de diseñar agentes inteligentes capaces de tomar decisiones óptimas en entornos con incertidumbre y complejidad combinatoria ha sido una cuestión central en inteligencia artificial desde sus orígenes. Los juegos de estrategia abstractos, en particular, presentan desafíos computacionales significativos que requieren el desarrollo de algoritmos sofisticados de búsqueda y evaluación. Hex es un representante paradigmático de esta clase de problemas: aunque sus reglas fundamentales son simples de comprender, el factor de ramificación elevado del árbol de juego y la necesidad de evaluar posiciones profundas hacen que la búsqueda exhaustiva sea computacionalmente prohibitiva incluso en tableros de tamaño moderado.

El presente proyecto se propone implementar y optimizar experimentalmente un jugador virtual competitivo para Hex mediante la exploración sistemática de variantes progresivas de Monte Carlo Tree Search bajo restricción temporal estricta. La estrategia investigativa comienza con MCTS básico, añade secuencialmente técnicas modernas como evaluación rápida de acciones (RAVE), reciclaje de árbol entre movimientos, paralelización, estrategias adaptativas según fase del juego, y análisis de tableros reducidos mediante detección de celdas muertas. Se evalúa críticamente el impacto de cada mejora en la calidad de decisión y el número de simulaciones alcanzables en el presupuesto temporal de 5 segundos por movimiento.

1. Descripción del Juego Hex

1.1. Fundamentales y Reglas del Juego

Hex es un juego de mesa combinatorio abstracto de información perfecta para dos jugadores, inventado por el matemático danés Piet Hein en 1942 y posteriormente redescubierto de forma independiente por el galardonado con el Premio Nobel John Nash en 1948 [5]. El juego se disputa en un tablero romboidal compuesto por celdas hexagonales dispuestas en un patrón regular. Los tableros estándar son de dimensión $n \times n$, siendo común en competiciones internacionales el uso de tableros de 11×11, aunque tableros más pequeños (5×5, 7×7) se emplean frecuentemente en experimentación algorítmica.

El objetivo del juego es conectar dos bordes opuestos del tablero con piezas del propio color. Específicamente, el jugador 1 intenta establecer una conexión ininterrumpida desde el borde izquierdo al borde derecho del tablero, mientras que el jugador 2 persigue conectar el borde superior al borde inferior [3]. Los dos jugadores alternan colocando una pieza de su color en una celda vacía del tablero. Una conexión válida consiste en una secuencia de piezas del mismo color tales que cada pieza es adyacente (en la geometría hexagonal estándar) a la

siguiente. La primera parte de Hex no posee tiradas de azar, sorpresas ni ocultamiento de información, clasificándola como un juego de información perfecta determinista.

1.2. Propiedades Matemáticas y Complejidad Computacional

Hex pertenece a la clase de juegos combinatorios abstractos estudiados extensivamente en teoría de juegos. Una propiedad fundamental del juego es que nunca puede terminar en empate: la topología del tablero hexagonal garantiza que en cualquier configuración final ocupada, exactamente uno de los jugadores habrá completado una conexión entre sus bordes designados. Esta propiedad, conocida como el "teorema de estrategia de robo" (strategy stealing argument), implica que para tableros grandes, el primer jugador posee una estrategia ganadora garantizada, aunque tal estrategia no ha sido computacionalmente determinada para tableros de competición estándar.

La complejidad computacional de Hex crece exponencialmente con el tamaño del tablero. El factor de ramificación medio presente en un árbol de juego hexagonal de dimensión n es aproximadamente $O(n^2)$, haciendo que la búsqueda exhaustiva resulte infactible para tableros de tamaño moderado. Un tablero 5×5 contiene 25 celdas y un máximo de 25 jugadas posibles, dando un árbol teórico de complejidad aproximada 25! sin aplicar técnicas de poda. Para un tablero 11×11 (121 celdas), la complejidad crece a magnitudes donde incluso algoritmos sofisticados con modernos procesadores requieren métodos heurísticos y probabilísticos.

1.3. Geometría Hexagonal y Representación del Tablero

El tablero de Hex utiliza la geometría de teselación hexagonal regular. Cada celda posee exactamente seis vecinos directos (a menos que la celda se encuentre en un borde o esquina del tablero). En la implementación computacional, la representación espacial del tablero es crítica para eficiencia algorítmica. La convención más común es el esquema *even-r*, donde las filas pares (fila 0, 2, etc.) tienen un desplazamiento horizontal diferente respecto a filas impares. Esta representación permite calcular los seis vecinos de una celda mediante un conjunto fijo de offsets de coordenadas, lo que acelera significativamente operaciones de búsqueda de caminos y evaluación de conectividad.

2. Algoritmos de Búsqueda

2.1. Minimax

2.1.1. Desarrollo Histórico

El algoritmo Minimax constituye uno de los pilares fundacionales de la inteligencia artificial en juegos, originándose en la teoría de juegos combinatorios. Su formulación moderna se debe a las contribuciones de Von Neumann, Turing y Shannon en los años 1950. La adicional de poda alfa-beta, desarrollada independientemente por varios investigadores a finales de los años 1960, representa un hito computacional al demostrar que podía reducirse significativamente el árbol de búsqueda explorado sin pérdida de optimalidad.

2.1.2. Principios Fundamentales

Minimax es un algoritmo de búsqueda exhaustiva que evalúa posiciones mediante construcción recursiva de un árbol de juego hasta una profundidad máxima especificada. El algoritmo asume un jugador *maximizador* que intenta lograr el mayor valor posible, y un jugador *minimizador* que persigue minimizar el valor obtenido por su oponente. En cada nodo del árbol, si la profundidad alcanzada es máxima, se evalúa la posición usando una función heurística de evaluación. De otro modo, el valor del nodo se computa recursivamente como el máximo de los valores de nodos hijos sucesores (en niveles maximizadores) o el mínimo (en niveles minimizadores).

2.1.3. Poda Alfa-Beta

La poda alfa-beta optimiza Minimax mediante observación de que en ciertos puntos del árbol de búsqueda pueden descartarse ramas completas sin evaluarlas explícitamente. Se mantienen dos variables globales, alfa (la mejor valoración que puede garantizar el jugador maximizador) y beta (la mejor valoración que puede garantizar el jugador minimizador). Cuando durante la búsqueda se detecta que beta $\leq$ alfa, se produce un *cutoff* y la rama actual se poda, ahorrando evaluaciones innecesarias.

El desempeño de alfa-beta depende crucialmente del orden en que se examinan los movimientos (move ordering). Si los movimientos son evaluados en orden de promesa decreciente (mejores primero), la cantidad de nodos podados es significativamente mayor.

2.2. Monte Carlo Tree Search

2.2.1. Contexto Histórico y Motivación

Monte Carlo Tree Search (MCTS) emerge a mediados de la década del 2000 como paradigma revolucionario para búsqueda en espacios de decisión combinatorios complejos. Los algoritmos tradicionales de juegos, como Minimax con poda alfa-beta, presentan dificultades crecientes en juegos con factor de ramificación muy alto, particularmente en Go, donde el factor de ramificación promedio es aproximadamente 250. MCTS dirigió la atención hacia métodos que combinaban muestreo aleatorio con construcción progresiva de un árbol de búsqueda, aprovechando la concentración de la distribución de probabilidades para explorar eficientemente espacios amplios.

2.2.2. Principios Fundamentales

MCTS opera mediante iteración repetida de cuatro fases bien definidas, como se describe en desarrollos posteriores en el dominio de Hex [3]:

La fase de **selección** comienza en la raíz del árbol y desciende mediante la aplicación de una política de selección (típicamente la fórmula UCT) hasta alcanzar un nodo no completamente explorado. La fórmula UCT (Upper Confidence bound for Trees) balancea exploración mediante el término de confianza superior y explotación mediante estimaciones de valor acumuladas.

En la fase de **expansión**, se agrega un nuevo nodo hijo al árbol, corresponde a un movimiento no previamente explorado desde la posición alcanzada en la fase anterior.

Durante la fase de **simulación**, desde el nodo recientemente expandido se ejecuta una política de juego (comúnmente aleatorio puro en variantes básicas) hasta alcanzar una posición terminal del juego. El resultado de esta simulación (victoria, derrota o empate) se registra.

Finalmente, en la fase de **retropropagación** (backpropagation), el resultado de la simulación se propaga hacia atrás a través del camino desde el nodo expandido hasta la raíz, actualizando estadísticas de visitas y recompensas acumuladas en cada nodo del camino.

2.2.3. Fórmula UCT y Balance Exploración-Explotación

La selección de nodos hijos se realiza tradicionalmente mediante la fórmula UCT, definida como:

$$\mathrm{UCT}(v) = \frac{Q(v)}{N(v)} + C\sqrt{\frac{\ln N(p(v))}{N(v)}} \tag{1}$$

donde $Q(v)$ denota el número acumulado de simulaciones ganadoras desde el nodo v, $N(v)$ es el número total de visitas al nodo, $N(p(v))$ es el número de visitas al nodo padre, y C es una constante de exploración ajustable. El primer término propicia explotación de nodos con alto rendimiento histórico, mientras el segundo término proporciona un bonus de exploración decreciente conforme un nodo es visitado más frecuentemente (permitiendo así revisar jugadas poco explotadas).

3. Implementación y Estrategias Experimentales

Durante el desarrollo de este proyecto se implementaron y probaron seis variantes progresivas del algoritmo MCTS, cada una incorporando mejoras algorítmicas específicas que permitieron evaluar el impacto de distintos componentes en la calidad del juego.

Se implementa RAVE (Rapid Action Value Estimation) que proporciona estimaciones tempranas de valor de acciones mediante estadísticas all-moves-as-first acumuladas durante playouts. Se exploran adicionalmente estrategias progresivas que adaptan dinámicamente parámetros de exploración según avanza la partida, así como reciclaje de árbol de búsqueda para reutilizar trabajo computacional entre movimientos sucesivos.

Ambas estrategias se complementan con heurísticas de dominio específico que detectan patrones de conectividad relevantes en Hex. Se implementan técnicas de análisis de celdas inferiores para poda del árbol de búsqueda, incluyendo la detección de celdas muertas, esquinas aisladas y regiones capturadas. Los patrones explorados incluyen análisis de ocupación de vecinos para garantizar que ciertos espacios no contribuyen a ninguna estrategia ganadora.

Se utilizó una estrategia de selección de jugadas que pondera el porcentaje de victorias (win rate) y la cantidad de visitas de cada nodo (visit count) mediante una constante `ALPHA` (fijado en 0.7 de forma experimental). Este enfoque es diferente al paradigma de selección encontrado en la bibliografía [1].

3.1. BasicMCTSPlayer: MCTS Convencional

La implementación básica constituye el caso de referencia fundamental, incorporando los cuatro pasos canónicos de MCTS sin mejoras adicionales. En el nivel de selección se utiliza la fórmula UCT con constante de exploración C = 0.1 [5]. Las simulaciones se ejecutan mediante playouts completamente aleatorios desde nodos hoja hasta posiciones terminales.

Ofrece simplicidad conceptual, bajo overhead computacional y comportamiento predecible y susceptible a análisis. Sin embargo, su convergencia lenta en espacios de búsqueda grandes hace que requiere un número significativamente mayor de simulaciones para alcanzar decisio-

nes confiables comparado con versiones mejoradas, y es propenso a explorar excesivamente el árbol sin explotarlo lo suficiente.

3.2. RecyclerMCTSPlayer: MCTS con Reciclaje de Árbol

Esta variante mejora el rendimiento temporal mediante la reutilización del árbol de búsqueda entre movimientos sucesivos (i. e. logra incorporar al análisis más simulaciones al reciclar las pertinentes de la exploración anterior). Después de que el contrincante realiza su movimiento, se detecta qué acción fue ejecutada comparando las dos configuraciones del tablero. Si esa acción existe como hijo del nodo raíz previamente explorado, la búsqueda continúa desde ese subtree, preservando todas las estadísticas acumuladas.

Aprovecha el trabajo computacional previo, reduce la latencia en movimientos posteriores y mantiene continuidad estratégica en la exploración; pero tiene algunas desventajas como complejidad adicional en la detección de movimientos opuestos, overhead de comparación de tableros y dependencia de que el árbol anterior sea reutilizable.

3.3. RAVEMCTSPlayer: MCTS con Evaluación Rápida de Acciones

Esta implementación incorpora la técnica RAVE (Rapid Action Value Estimation) mediante estadísticas all-moves-as-first (AMAF). Cada nodo mantiene dos conjuntos de estadísticas: las estadísticas UCT convencionales y las estadísticas AMAF que acumulan resultados de simulaciones donde una acción determinada fue jugada en cualquier posición, independientemente de su posición en el árbol. La selección de hijos combina ambas estimaciones mediante una fórmula de decaimiento que transita progresivamente desde dominancia RAVE hacia dominancia UCT conforme aumentan las visitas.

El parámetro `exploration_c` fue calibrado experimentalmente a 0,5, valor que supera las recomendaciones bibliográficas [5] para Hex (posiblemente debido a la restricción temporal de 5 segundos del proyecto); y el `rave_bias` fue fijado a 0,00025 como era sugerido [5]. Mostró como predijo la teoría una convergencia más rápida en fases tempranas de exploración, estimaciones de valor más confiables con menos simulaciones y un equilibrio mejorado entre exploración y explotación. Provocó un aumento de la complejidad temporal y espacial en el mantenimiento de estadísticas duales y de riesgos de sobrecapacitación a patrones locales si RAVE no decae adecuadamente.

3.4. ProgressiveMCTSPlayer: MCTS con Estrategias Progresivas Adaptativas

Esta variante incorpora ajuste dinámico de parámetros según la fase del juego. Se divide la partida en tres fases: apertura, medio juego y final, cada una con parámetros diferentes de `exploration_c` y `trave_bias` [6]. Adicionalmente, durante la fase final se incorporan heurísticas de simulación que evalúan posiciones según distancia a conexión ganadora y construcción de 2-bridges [2] [3], resultando en playouts guiados que detectan amenazas inmediatas.

En experimentos realizados se observó que la combinación de poda progresiva (progressive unpruning) con sesgo progresivo resultaba en pérdidas masivas de tasas de victoria contra RAVEMCTSPlayer. Se determinó que este comportamiento se debía a incompatibilidades en el ajuste de parámetros y se decidió mantener solamente el sesgo progresivo. Aún con el aumento en la tasa de ganancias manteniendo esta única estrategia, no se logró superar los resultados de RAVEMCTSPlayer.

Esta estrategia se adapta a dinámicas de juego cambiantes, explota de forma más capaz la información del endgame mediante heurísticas especializadas y muestra potencial para mayor eficiencia temporal. Sin embargo, presenta mayor complejidad paramétrica, un riesgo (que en este proyecto no pudo superarse) de ajustes contraproducentes que desfavorezcan ciertas fases. Necesitaría una calibración exhaustiva para cada tamaño de tablero, posiblemente usando técnicas de aprendizaje de máquinas.

3.5. ParallelizedMCTSPlayer: MCTS Paralelizado con Sincronización Ordenada

Esta implementación distribuye la búsqueda de árbol entre múltiples hilos trabajadores (*worker threads*) que exploran simultáneamente un árbol compartido. Cada nodo del árbol incorpora candados (*locks*) de sincronización granular: un candado para las estadísticas de visitas y ganancias, y otro para la expansión de hijos. Este diseño minimiza la contención de locks comparado con otras estrategias de sincronización global. Se emplean 4 threads paralelos que compiten por acceso al árbol. La integración con RAVE proporciona combinación similar de estadísticas como en RAVEMCTSPlayer, pero ejecutada en contexto de exploración multiconcurrente.

Con ella se aprovechan arquitecturas de procesador multicore que pueden generar mayor número de simulaciones por unidad de tiempo en sistemas con múltiples núcleos disponibles; aunque la sobrecarga de sincronización puede superar beneficios en sistemas con limitaciones del modelo de ejecución de Python (*GIL - Global Interpreter Lock*) y la complejidad se ve sustancialmente aumentada. En el intérprete de referencia se presentó una ligera degradación

de rendimiento respecto a RAVEMCTSPlayer.

3.6. ReducedBoardMCTSPlayer: MCTS con Tableros Reducidos mediante Fillin

Esta estrategia experimental implementa un análisis de celdas inferiores (*fillin*) que identifica celdas muertas y regiones sin interés estratégico [3]. Dos enfoques fueron investigados y comparados:

- Fillin Dinámico Periódico: Análisis de fillin periódicamente durante el juego (cada $n/2$ movimientos aproximadamente). Cuando se detectaban celdas muertas y esquinas aisladas, se marcaban como ocupadas por el jugador correspondiente, reduciendo el tablero efectivo. Los patrones detectados incluían celdas completamente rodeadas por piezas de un jugador y esquinas aisladas sin rutas de escape. La aplicación dinámica durante la partida generaba overhead significativo que neutralizaba los beneficios: aproximadamente 30-40 por ciento de reducción de simulaciones totales por movimiento. El experimento reveló que no es viable bajo restricción temporal de 5 segundos.

- Fillin Fijo al Inicio: Análisis de fillin aplicado una única vez al inicializar el tablero, detectando celdas muertas con los mismos patrones locales (ocupación uniforme de vecinos, esquinas aisladas). Esta aproximación transfiere el overhead de detección al inicio, evitando análisis repetitivos durante la búsqueda. Los resultados experimentales demostraron que el fillin fijo inicial supera significativamente el enfoque dinámico, generando un número de simulaciones por movimiento similar al de otros enfoques. Sin embargo, el rendimiento de ganancia no superó a RAVEMCTSPlayer sin fillin.

Esta estrategia teóricamente promete una mejora potencial en fases de endgame mediante estructuras de tablero reducido y es compatible con arquitecturas de árbol de búsqueda existentes. Sin embargo, en la práctica no ofreció beneficio comparado con RAVEMCTSPlayer puro bajo restricción temporal de 5 segundos. Pudieran explorarse otras estructuras rellenables para lograr una mejoría.

3.7. Noobex: Jugador Final

Después de una experimentación exhaustiva, se seleccionó RAVEMCTSPlayer como estrategia base del jugador final, pero reimplementado con paralelización para explotar arquitecturas multicores y teniendo en cuenta que no hubo un decrecimiento apreciable en el

porcentaje de victorias contra el algoritmo lineal. El jugador resultante, denominado SmartPlayer según los requerimientos del proyecto (pero bautizado, desde el cariño de su creador, como *Noobex*), mantiene el algoritmo de ParallelMCTSPlayer con heurísticas de endgame integradas, activadas según la cantidad de piezas presentes en el tablero (umbral definido: $n * \sqrt{n}$). Teóricamente mostrará todo su potencial cuando el ambiente de ejecución permite el desbloqueo del GIL (por ejemplo, en compiladores/intérpretes alternativos de Python o implementaciones compiladas de C++).

4. Análisis del Jugador Final Noobex

4.1. Descripción de SmartPlayer

SmartPlayer implementa Monte Carlo Tree Search paralelizado con mejoras RAVE y reciclaje de árbol entre movimientos. La arquitectura comprende cuatro worker threads que ejecutan iteraciones MCTS concurrentes sobre un árbol compartido sincronizado mediante locks. En cada iteración, se navega el árbol udsnndo la fórmula UCT+RAVE combinada (influenciada con heurística de detección de puentes en endgame), se expande un nodo no explorado, se ejecuta un playout aleatorio (guiado por heurísticas en endgame) hasta conclusión del juego, y se retropropaga el resultado actualizando estadísticas en todos los nodos del camino.

El playout está optimizado mediante cálculo rápido de posiciones vacías utilizando Union-Find (*disjoint sets union*), estructura que acelera operaciones de verificación de ganancia y mantenimiento de conectividad mediante operaciones de unión eficientes. Para utilizarla, se diseñó un tablero auxiliar (*BoardOptimized*) que optimiza la ejecución del algoritmo al ofrecerle información de forma rápida y organizada.

La selección de las jugadas es realizada de la misma forma que en el resto de las estrategias implementadas: $max_{node}\{(1 - \texttt{ALPHA}) * \texttt{win_rate} + \texttt{ALPHA} * \texttt{visit_rate}\}$, con `visit_rate` definido como la razón entre la cantidad de visitas al nodo durante la exploración y la cantidad de simulaciones realizadas (iteraciones del algoritmo).

El endgame necesario para la activacion de las heurísticas es activado a través de la cantidad de nodos en el tablero, con un umbral definido en $n * \sqrt{n}$ posiciones ocupadas. Fuera de esto, usa conocimiento de juego para detectar jugadas obvias como posiciones ganadoras tanto para el jugador como para el oponente (con el objetivo de bloquearla) y sugerencia de jugadas iniciales, cuando la selección es práctiamente aleatoria.

4.2. Ventajas

La paralelización permite teóricamente aprovechar múltiples núcleos de procesador para incrementar el número de simulaciones ejecutadas en el presupuesto temporal fijo de 5 segundos. Noobex alcanza tasas de ganancia de aproximadamente 70 por ciento contra RecyclerMCTSPlayer, indicando que el algoritmo base proporciona decisiones de calidad substancial. El reciclaje de árbol permite reutilizar trabajo computacional, mejorando la eficiencia entre movimientos sucesivos. Las heurísticas de detección temprana (winning move, blocking move y opening move inmediatos) evitan búsquedas costosas cuando existen movimientos óptimos obvios.

4.3. Limitaciones

En el intérprete de referencia de Python (CPython), el Global Interpreter Lock (GIL) serializa la ejecución de código bytecode, eliminando los beneficios teóricos de paralelización. Esto hace que tenga un redimiento similar y ligeramente inferior (debido a la complejidad añadida) al RAVEMCTSPlayer.

4.4. Recomendación de Uso

Se recomienda utilizar SmartPlayer (*Noobex* para los amigos) en ambientes donde CPython con GIL no sea la plataforma de ejecución, tales como PyPy con jit, Jython, o compiladores a máquina nativa. En tales contextos, la paralelización debiera proporcionar speedups tangibles. Para CPython con restricción temporal de 5 segundos, RAVEMCTSPlayer secuencial representa una opción ligeramente superior, aunque las diferencias son prácticamente despreciables.

Se proporciona SmartPlayer como solución principal por su promesa teórica con intérpretes más bondadosos, pero queda documentado este comportamiento en CPython nativo para referencia futura.

5. Resultados Experimentales

Los resultados muestran que la evolución progresiva del algoritmo MCTS base hacia versiones mejoradas genera incrementos consistentes en rendimiento en algunos lugares, mientras en otros presenta un freno:

- RecyclerMCTSPlayer mejora notablemente la tasa de ganancia y el número de simulaciones promedio, demostrando la utilidad substancial del reciclaje de árbol entre mo-

vimientos sucesivos.

- RAVEMCTSPlayer con reciclaje de árbol y heurísticas de endgame alcanza la mejor tasa de ganancia contra oros jugadores, indicando que la evaluación rápida de acciones (RAVE) proporciona mejora significativa al acelerar convergencia en simulaciones tempranas.

- ProgressiveMCTSPlayer no demostró mejora adicional clara bajo los parámetros experimentados. Se conjetura que la incompatibilidad entre poda progresiva y sesgo progresivo en configuración nivelada neutralizó los beneficios teóricos. El sesgo progresivo por su cuenta tampoco mejoró los resultados anteriores, lo que sugiere que hace falta un ajuste más profundo de los parámetros utilizados.

- ReducedBoardMCTSPlayer experimentó con dos enfoques de fillin pero no alcanzó el rendimiento de RAVEMCTSPlayer. Las causas de este comportamiento pudieran analizarse con mayor profundidad en el futuro, dado que es una de las estrategias implementadas más prometedoras.

- ParallelizedMCTSPlayer no genera beneficios prácticos debido al control Global Interpreter Lock (GIL) del intérprete de Python utilizado.

6. Investigación Futura

Aunque este proyecto proporciona un framework experimental sólido para desarrollar MCTS en Hex, varias avenidas de investigación importantes permanecen sin exploración debido a restricciones temporales y de alcance:

- Análisis completo de celdas inferiores: MoHex, ganador del Computer Olympiad 2009, implementa 250+ patrones locales para detección de celdas inferiores, comparado con los patrones simplificados (celdas completamente rodeadas, esquinas aisladas) que se intentaron en ReducedBoardMCTSPlayer. Una investigación futura debería implementar jerarquía de patrones multi-nivel siguiendo la taxonomía de Arneson et al. [3], incluyendo puentes básicos (dos piezas adyacentes), 4-3-2 templates, estructuras Ziggurats, y composiciones más complejas.

- Integración de búsqueda de conexiones virtuales: Las conexiones virtuales representan un mecanismo fundamental en Hex estratégico que captura relaciones de dominancia entre grupos sin conexión directa [3]. Cazenave [5] demostró aumento de 11 por ciento en tasa de ganancia mediante integración de búsqueda de conexiones virtuales. Una

investigación futura debería implementar H-search (Hayward search) para la identificación eficiente de virtual connections y virtual semiconnections, integrable en fase de selección de MCTS para poda automática de movimientos que no contribuyen a amenazas críticas o jugadas ganadoras.

- Solvers especializados de endgame: Durante la fase final del juego, cuando la ocupación es alta y el número de movimientos restantes es moderado, técnicas de búsqueda exhaustiva especializadas podrían superar la exploración probabilística de MCTS. Podría implementarse un solver de endgame que detecte cuando la ocupación supera un umbral (por ejemplo, 70-80 por ciento del tablero) y transite a búsqueda minimax con heurísticas de evaluación especializadas para posiciones finales. La literatura sugiere que una combinación híbrida MCTS + minimax en fase final puede superar ambas estrategias independientes, lo que fue comprobado en juegos como Kalah, Breakthrough, and Lines of Action [4].

- Implementación de Minimax para tableros pequeños: Fue una de las ideas principales al comenzar la implementación de los jugadores, pero que no pudo explorarse antes de la fecha de entrega. El determinismo que ofrecen los algoritmos Minimax con buenas heurísticas de completamiento de partidas, permiten garantizar excelentes resultados en espacios de búsqueda reducidos. Quedaría definir un umbral de cuando un juego es considerado lo suficientemente pequeño (se estima que $n \leq 7$) y si realmente ofrece mejoras en jugadas frente a un MCTS fortalecido.

- Experimentación multicore real: La limitación del Global Interpreter Lock en CPython fue identificada como un cuello de botella crítico para nuestro SmartPlayer paralelizado. Investigación futura debería evaluar rendimiento en plataformas alternativas como PyPy con JIT que permite parallelismo real. También pudiera pensarse en una implementación nativa en C++ o C con enlaces Python. Se predice que una verdadera paralelización podría permitir un incremento de 2 a 3 veces en simulaciones en el mismo tiempo, potencialmente superando definitivamente el RAVEMCTSPlayer secuencial.

- Ajuste profundo de los valores de los parámetros: Todos los jugadores experimentales fueron calibrados empíricamente con un número limitado de iteraciones de ajuste. Análisis rigurosos de parámetros críticos (`exploration_c`, `rave_bias`, `phase_thresholds`) mediante grid search o métodos de optimización Bayesianos podría revelar configuraciones significativamente superiores. En particular, se observó que `exploration_c` = 0,5 diverge de recomendaciones bibliográficas (0,0 para Hex con RAVE) [5], sugiriendo que la situación de los parámetros bajo restricción de 5 segundos y sin solvers iniciales

complejos difiere de la explorada en literatura.

7. Conclusiones

Este proyecto presenta un análisis sistemático comparativo de estrategias de búsqueda en el juego Hex bajo restricción temporal de 5 segundos por movimiento. Se construyó una jerarquía experimental que avanza desde Monte Carlo Tree Search básico hacia versiones aumentadas con RAVE, tree recycling, paralelización, estrategias progresivas, heurísticas y análisis de tableros reducidos. Los resultados experimentales demuestran que para el régimen de restricción temporal de este proyecto, el algoritmo MCTS con RAVE y tree recycling representa el óptimo viable. Quedan propuestas las posibles mejoras a explorar, pero se presneta a Noobex como jugador de Hex competente.

Referencias

[1] Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

[2] Kunčarová, L. (2024). *Artificial Intelligence for the Hex Game*. Bachelor thesis, Department of Theoretical Computer Science and Mathematical Logic, Charles University, Prague.

[3] Arneson, B., Hayward, R. B., & Henderson, P. (2010). Monte Carlo Tree Search in Hex. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(4), 251–257.

[4] Lanctot, M., Winands, M. H. M., Pepels, T., & Sturtevant, N. R. (2014). Monte Carlo Tree Search with Heuristic Evaluations using Implicit Minimax Backups. In *Proceedings of the 6th International Symposium on Combinatorial Game Theory* (CSCG 2014). Department of Knowledge Engineering, Maastricht University.

[5] Cazenave, T., & Saffidine, A. (2009). Monte-Carlo Hex. In *Proceedings of the Computer Games Workshop* (CG 2009). LAMSADE, Université Paris-Dauphine and École Normale Supérieure de Lyon.

[6] Chaslot, G. M. J-B., Winands, M. H. M., van den Herik, H. J., Uiterwijk, J. W. H. M., & Bouzy, B. (2008). Progressive Strategies for Monte-Carlo Tree Search. In *Proceedings of the 2008 IEEE Symposium on Computational Intelligence and Games*. MICC-IKAT, Universiteit Maastricht and Centre de Recherche en Informatique de Paris 5.